

Курс Системотехника

Борис Карышев

15 октября 2025 г.

Содержание

1	Описание	3
2	ДЗ 1	4
2.1	Условие	4
2.2	Решение	4
2.2.1	Наивное решение	4
2.2.2	Улучшенное решение	4
2.3	Проверка корректности	5
3	ДЗ 2	6
3.1	Условие	6
3.2	Решение	6
3.3	Проверка корректности	6

1 Описание

Данная документация идет к репе: https://github.com/BorisKaryshev/verilog_iu4

Она описывает код по верилогу, написанный в рамках обучения в магистратуре на кафедре ИУ4 (2025-2027).

Выполнил Карышев Борис Владимирович, группа ИУ4-11М. Осень, 2025-2026 учебный год.

2 ДЗ 1

2.1 Условие

На вход модуля каждый так подается число x .

Необходимо вычислить:

$$y_{n+1} = a \bullet y_n + b \bullet x_n$$

Операция умножения занимает 2 такта!!!

Все числа 8-ми битные, беззнаковые. y_0 , a и b - константы, зашитые в железо (в код верилога). Значения x могут поступать очень долго (бесконечно). При этом переполнение переменной y допускается.

2.2 Решение

2.2.1 Наивное решение

Наивное решение заключается в том, чтобы просто посчитать формулу. Однако в рамках данной задачи подобное решение не подойдет ввиду заданных ограничений.

Проблема заключается в том, что для вычисления следующего значения нужно предыдущее, которое будет вычислено через такт. Таким образом, задержка вычислений будет возрастать каждый такт на 1 такт. Так же, необходимо предусмотреть память, в которой будут храниться x для ещё не вычисленных y_n , что невозможно реализовать, так как по условию, поток x бесконечен.

2.2.2 Улучшенное решение

Преобразуем заданное для вычислений уравнение, подставив в y_{n+1} значение от y_n :

$$y_{n+1} = a \bullet y_n + b \bullet x_n = a \bullet (a \bullet y_{n-1} + b \bullet x_{n-1}) + b \bullet x_n = a^2 \bullet y_{n-1} + ab \bullet x_{n-1} + b \bullet x_n$$

При этом, в полученном выражении a^2 и ab - константы, вычисление которых можно вынести за пределы алгоритма. Таким образом, они будут "зашиты" в железо так же, как и константы a и b .

Полученное выражение $y_{n+1} = f(y_{n-1}, x_{n-1}, x_n)$ будет вычисляться так же, как и заданное в условии, за 2 такта. При этом, выходная последовательность y разделяется на две независимые друг от друга подпоследовательности: четную и нечетную. Каждая из которых может вычисляться независимо друг от друга, т. е. параллельно. При этом, вычислять следующий элемент необходимо каждый четный или нечетный x соответственно.

Таким образом, полученное уравнение вычисляется за 2 такта, при частоте вызовов в 2 такта.

Алгоритм состоит из следующих основных этапов:

1. Получаем x_1 и вычисляем y_1 через формулу из условия.
2. Получив x_2 запускаем вычисления четной подпоследовательности по полученной формуле.
3. Получив x_3 запускаем вычисления нечетной подпоследовательности по полученной формуле.
4. Продолжаем выполнять 2 предыдущих этапа.

2.3 Проверка корректности

Ниже приведен скриншот выполнения кода в ПО gtkwave (рисунок 1). При этом разработанный модуль не выводит значение нулевого y .

На вход подаются следующие значения (таблица 1).

X	Got	Expected
0	0	0
1	0	0
2	1	1
3	3	3
4	6	6
7	11	11
12	18	18
-	30	30

Таблица 1: Демонстрационные данные

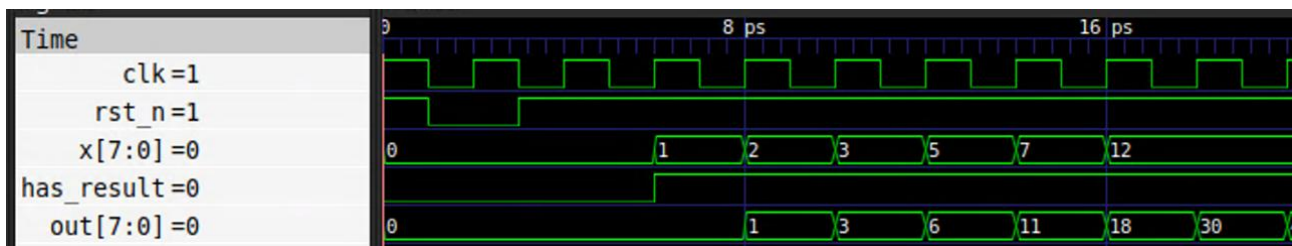


Рис. 1: Скриншот из GTKWave

Так же мной были написаны автоматические тесты, валидирующие корректность разработанной программы.

Код тестов лежит по пути: `sem3/test_benches/main.cpp`

3 ДЗ 2

3.1 Условие

Нужно написать модуль - приемник данных, передаваемых по протоколу SPI. Передаются:

- 1 байт - команда
- 2 байт - адрес
- 4 байт - данные

Необходимо выдать данные на выход параллельно.

3.2 Решение

На выходе 3 регистра, которые заполняются последовательно. Для соотв бита сдвигаем регистр, устанавливая текущий бит.

Данные передаются в порядке из условия: команда, адрес, данные.

Байты передаются по соглашению: MSB.

3.3 Проверка корректности

Ниже приведен скриншот выполнения кода в ПО gtkwave (рисунок 2). На вход прдаются следующие значения:

- command = 38
- address = 238
- data = 10038



Рис. 2: Скриншот из GTKWave

Так же мной были написаны автоматические тесты, валидирующие корректность разработанной программы на 1.000.000 случайных данных.

Код тестов лежит sem4/test_benches/main.cpp